

CSE 220 — Sampling

Five More Online-Style Assignments from Lecture 5

1 What the lecture covers, and what is left

Lecture 5 (*Sampling*, 47 slides) runs from the impulse-train picture of sampling all the way to the inverse DFT. The two assignments you already have use two of its topics. Here is the whole map, with the gaps marked.

Topic	Slides	Status
Ambiguity: many signals, one sample set	5	a.md
Nyquist theorem, aliasing, undersampling	19–22	a.md
Zero-order hold, sinc droop	28–31	b.md
Sampling as multiplication by an impulse train	6, 13–15	assignment C
Spectrum replication: copies every ω_s	16–17, 20–21	assignment C
The folding rule: where a tone lands	19–22	assignment D
Ideal reconstruction: LPF and sinc interpolation	24–27	assignment E
Comparing the three reconstruction filters	36	assignment E
Linear interpolation (first-order hold)	32–35	assignment F
The Nyquist rate as a threshold	19–20	assignment G
Duality; multiplication $\leftrightarrow$ convolution	11–12	derivation only
Uncertainty principle, unavoidable leakage	38	derivation only
Connection to the DFT: DTFT, bins, $X[k] = Na_k$	37–47	not sampling

Two rows are left without an assignment because they are proofs rather than things you can measure with a pass/fail script. The last block is left out for a different reason: slides 37–47 are about how the DFT relates to the Fourier series, which is a DFT topic that happens to be taught at the end of the sampling lecture. Since the online is strictly on sampling, every assignment below stays inside the sampling story — impulse-train sampling, spectral copies, the Nyquist threshold, aliasing, and the three reconstruction filters.

Each assignment follows the same shape as the two you have: a short background, two functions with their bodies removed, a `main()` that is already written, and a table that compares a *predicted* value against a *measured* one. Every expected-output block in this document was produced by actually running the answer code.

The five files: `copies.py`, `folding.py`, `sincrecon.py`, `linterp.py`, `nyquist.py`.

Contents

1 What the lecture covers, and what is left	1
2 Assignment C — Sampling Copies the Spectrum	2

3	Assignment D — Where Does the Tone Land?	5
4	Assignment E — The Ideal Filter Does Not Droop	8
5	Assignment F — Straight Lines Droop Twice as Hard	11
6	Assignment G — The Nyquist Threshold	14

2 Assignment C — Sampling Copies the Spectrum

Background

Sampling is multiplication by an impulse train, and the impulse train's spectrum is itself a train of impulses spaced $\omega_s = 2\pi/T$ apart. Convolution with an impulse does nothing but place a copy, so the spectrum of the sampled signal is

$$X_p(j\omega) = \frac{1}{T} \sum_{k=-\infty}^{+\infty} X(j(\omega - k\omega_s)),$$

one copy of the original spectrum at every multiple of the sampling rate. This single picture is the whole Nyquist theorem: if the copies stay apart you can filter one out, and if they overlap you cannot.

A single cosine at frequency f has just two spectral lines, at $\pm f$. After sampling, those two lines are copied to every multiple of f_s , so lines appear at $|kf_s \pm f|$ for every integer k — and every one of them has exactly the same height. That is what you will build and check.

Your task

The starter file contains two functions with their bodies removed. Fill in both. `main()` and `tone_amplitude()` are provided; do not modify them.

Part 1 — `copy_frequencies(f, fs, fmax)` Return a sorted list of every positive frequency up to and including `fmax` at which a copy of the line appears, that is every value $|kf_s \pm f|$ that falls in $(0, f_{\max}]$.

Part 2 — `copy_spread(f, fs, upsample, duration, fmax)` Build the impulse-sampled signal on a fine grid running at `upsample * fs`: an array of zeros with the sample value $\cos(2\pi f n / f_s)$ placed at every `upsample`-th position. Measure the amplitude at each frequency from Part 1 with `tone_amplitude`, and return the difference between the largest and the smallest of those amplitudes. If the copies really are identical this is zero.

Starter file — `copies.py`

```
"""Sampling Copies the Spectrum.

Complete the two functions marked TODO. Do not modify anything below
the divider. Run with: python copies.py
"""

import numpy as np

def copy_frequencies(f, fs, fmax):
    """Every positive frequency <= fmax holding a copy of the line at f.

    These are the values |k*fs +- f| for integer k, sorted ascending.
    """
    raise NotImplementedError

def copy_spread(f, fs, upsample, duration, fmax):
    """Largest minus smallest copy amplitude in the sampled spectrum.

    Builds the impulse-sampled signal on a grid running at upsample*fs:
```

```

zeros everywhere except the sample values  $\cos(2\pi f n / fs)$ , which sit
at every upsample-th position. Measures the amplitude at each
frequency from copy_frequencies (using tone_amplitude, second
parameter upsample * fs) and returns max - min.
"""
raise NotImplementedError

# -----
# Everything below is provided. Do not modify.
# -----

def tone_amplitude(x, fs_fine, f):
    """Amplitude of the component of x at frequency f, via the DFT."""
    n = len(x)
    spectrum = np.fft.rfft(x)
    freqs = np.fft.rfftfreq(n, 1 / fs_fine)
    return 2 * np.abs(spectrum[int(np.argmax(np.abs(freqs - f)))] / n

SAMPLE_RATE = 1000      # Hz
UPSAMPLE = 100          # fine-grid steps per sample period
DURATION = 0.1          # seconds
FMAX = 3000             # Hz, how far up to look
TOLERANCE = 1e-9

TEST_FREQS = [50, 100, 300, 450]  # Hz

def main():
    print(f'f (Hz):>8} {'copies':>8} {'max-min amplitude':>20}  result")
    print("-" * 52)

    failures = 0
    for f in TEST_FREQS:
        copies = copy_frequencies(f, SAMPLE_RATE, FMAX)
        spread = copy_spread(f, SAMPLE_RATE, UPSAMPLE, DURATION, FMAX)

        ok = spread < TOLERANCE
        failures += not ok
        print(f"{f:>8} {len(copies):>8} {spread:>20.2e}  "
              f"{'all equal' if ok else 'UNEQUAL'}")
        print(f"{'':>8} at {'', '.join(str(c) for c in copies)} Hz")

    print("-" * 52)
    if failures:
        print(f"{failures} of {len(TEST_FREQS)} case(s) had unequal copies.")
    else:
        print("Every copy has the same height at every frequency.")

if __name__ == "__main__":
    main()

```

Expected output

f (Hz)	copies	max-min amplitude	result
50	6	1.73e-18	all equal
at 50, 950, 1050, 1950, 2050, 2950 Hz			

```

100      6      3.47e-18  all equal
    at 100, 900, 1100, 1900, 2100, 2900 Hz
300      6      0.00e+00  all equal
    at 300, 700, 1300, 1700, 2300, 2700 Hz
450      6      0.00e+00  all equal
    at 450, 550, 1450, 1550, 2450, 2550 Hz

```

Every copy has the same height at every frequency.

Answer

```

def copy_frequencies(f, fs, fmax):
    out = set()
    k = 0
    while k * fs - f <= fmax:
        for c in (k * fs - f, k * fs + f):
            if 0 < c <= fmax:
                out.add(c)
        k += 1
    return sorted(out)

def copy_spread(f, fs, upsample, duration, fmax):
    N = int(duration * fs)
    impulses = np.zeros(N * upsample)
    impulses[::upsample] = np.cos(2 * np.pi * f * np.arange(N) / fs)
    amps = [tone_amplitude(impulses, upsample * fs, c)
            for c in copy_frequencies(f, fs, fmax)]
    return float(max(amps) - min(amps))

```

Why the loop stops where it does. Since $kf_s + f$ is always larger than $kf_s - f$, once $kf_s - f$ passes $f_{\max}$ both candidates are out of range and nothing further can qualify. Using a `set` removes the duplicate that appears when f is exactly $f_s/2$.

Why `impulses[::upsample]` is the sampled signal. The impulse-sampled signal $x_p(t)$ is zero everywhere except at the sample instants nT , where it carries the weight $x(nT)$. On a grid that is `upsample` times finer, the sample instants are exactly the positions 0, `upsample`, `2×upsample`, ... — which is what the slice `[::upsample]` selects. Everything between them stays zero.

Why the spread comes out at 10^{-18} (that is, exactly zero). The fine array has $N \times \text{upsample}$ points but only N of them are nonzero, and they are evenly spread. Working the DFT sum through, the value at fine bin k depends only on $k \bmod N$ — so the spectrum repeats exactly every N bins, which on this grid is exactly every f_s hertz. The copies are not approximately equal, they are the same numbers. What you see is pure floating-point dust.

3 Assignment D — Where Does the Tone Land?

Background

Sampling copies the spectrum to every multiple of f_s . Turn that around and you get a practical rule: a tone at any frequency f , no matter how high, shows up inside the band $[0, f_s/2]$ at one specific place, and the sampled data cannot tell it apart from a real tone sitting there.

Take f modulo f_s to get r , the position within one period. If r is below $f_s/2$ the tone appears at r ; if it is above, the copy coming down from f_s arrives first, and the tone appears at $f_s - r$ instead. The frequency axis is effectively folded at $f_s/2$, which is why $f_s/2$ is called the folding frequency.

Your task

The starter file contains two functions with their bodies removed. Fill in both. `main()` is already written; do not modify it.

Part 1 — `baseband_alias(f, fs)` Return the frequency in $[0, f_s/2]$ at which a tone of frequency f appears after sampling at rate fs . Frequencies above Nyquist are allowed; that is the point.

Part 2 — `peak_frequency(f, fs, duration)` Sample $\cos(2\pi ft)$ at rate fs using $t = n/f_s$ for $n = 0 \dots N-1$ with $N = \text{int}(\text{duration} * fs)$, take the real DFT, and return the frequency of the bin with the largest magnitude. Use `np.fft.rfft` and `np.fft.rfftfreq`.

Starter file — folding.py

```
"""Where Does the Tone Land: the folding rule.

Complete the two functions marked TODO. Do not modify anything below
the divider. Run with: python folding.py
"""

import numpy as np

def baseband_alias(f, fs):
    """The frequency in [0, fs/2] where a tone at f appears after sampling.

    f may be above the Nyquist frequency.
    """
    raise NotImplementedError

def peak_frequency(f, fs, duration):
    """Frequency of the largest DFT bin of cos(2*pi*f*t) sampled at fs.

    Sample times are t = n/fs for n = 0 ... N-1, N = int(duration * fs).
    Use np.fft.rfft and np.fft.rfftfreq.
    """
    raise NotImplementedError

# -----
# Everything below is provided. Do not modify.
# -----

SAMPLE_RATE = 1000      # Hz
```

```

DURATION = 0.1          # seconds
TOLERANCE = 1e-9

TEST_FREQS = [50, 450, 550, 950, 1450, 2300]  # Hz

def main():
    print(f"{'f (Hz)':>8} {'f/fs':>7} {'predicted':>11} {'measured':>10} "
          f"{'|error|':>10}   region")
    print("-" * 62)

    failures = 0
    for f in TEST_FREQS:
        predicted = baseband_alias(f, SAMPLE_RATE)
        measured = peak_frequency(f, SAMPLE_RATE, DURATION)
        error = abs(predicted - measured)

        failures += error >= TOLERANCE
        region = "baseband" if f < SAMPLE_RATE / 2 else "aliased"
        print(f"{'f':>8} {'f / SAMPLE_RATE':>7.2f} {'predicted':>11.1f} "
              f"{'measured':>10.1f} {'error':>10.2e}   {region}")

    print("-" * 62)
    if failures:
        print(f"{failures} of {len(TEST_FREQS)} case(s) landed somewhere else.")
    else:
        print("Every tone landed exactly where the folding rule predicts.")

if __name__ == "__main__":
    main()

```

Expected output

f (Hz)	f/fs	predicted	measured	error	region
50	0.05	50.0	50.0	0.00e+00	baseband
450	0.45	450.0	450.0	0.00e+00	baseband
550	0.55	450.0	450.0	0.00e+00	aliased
950	0.95	50.0	50.0	0.00e+00	aliased
1450	1.45	450.0	450.0	0.00e+00	aliased
2300	2.30	300.0	300.0	0.00e+00	aliased

Every tone landed exactly where the folding rule predicts.

Answer

```

def baseband_alias(f, fs):
    r = f % fs
    return min(r, fs - r)

def peak_frequency(f, fs, duration):
    N = int(duration * fs)
    x = np.cos(2 * np.pi * f * np.arange(N) / fs)
    spectrum = np.abs(np.fft.rfft(x))
    freqs = np.fft.rfftfreq(N, 1 / fs)
    return float(freqs[int(np.argmax(spectrum))])

```

Why two lines are enough for Part 1. `f % fs` throws away all the whole copies of f_s , leaving a number r between 0 and f_s . Now there are only two candidates inside the band: the line at r itself, and the line at $f_s - r$ that belongs to the copy coming down from the next multiple. Whichever is smaller is the one inside $[0, f_s/2]$, and `min` picks it.

Reading the table. 550 Hz and 450 Hz produce the same answer, and so do 950 Hz and 50 Hz. That is exactly the ambiguity from `alias.py`, seen from the other side: there you were given f and asked for its partner, here you are given a partner and asked where it lands.

Why the measurement is exact. With $N = 100$ samples at 1000 Hz the bins are 10 Hz apart, and every folded frequency in the table (50, 300, 450) is a multiple of 10. The tone sits precisely on a bin, so `argmax` finds it with no rounding at all — hence the `0.00e+00` error column.

4 Assignment E — The Ideal Filter Does Not Droop

Background

The zero-order hold attenuates high frequencies, and the amount is $|\text{sinc}(f/f_s)|$. That raises the obvious question: attenuated compared to *what*? The answer is the ideal reconstruction filter, the brick-wall low-pass filter of slides 24–27, whose gain is perfectly flat across the whole passband.

Its impulse response is

$$h(t) = \text{sinc}\left(\frac{t}{T}\right), \quad \text{so} \quad x_r(t) = \sum_{n=-\infty}^{+\infty} x(nT) \text{sinc}\left(\frac{t - nT}{T}\right).$$

Each sample is replaced by a sinc centred on it, and the sines are added up. This is the Whittaker–Shannon interpolation formula. It is not realisable — the sum needs infinitely many samples and the sinc never stops — but with a long enough record it is very accurate away from the two ends, which is what you will confirm.

Your task

The starter file contains two functions with their bodies removed. Fill in both. `main()` and `tone_amplitude()` are provided; do not modify them.

Part 1 — `sinc_interpolate(samples, fs, t)` Given sample values taken at rate `fs` (so sample n sits at time n/f_s), return the reconstructed signal evaluated at every time in the array `t`. Note that $\text{sinc}((t - nT)/T) = \text{sinc}(f_s t - n)$, which is easier to vectorise.

Part 2 — `measured_ideal_gain(f, fs, oversample, duration)` Sample $\cos(2\pi f t)$ at rate `fs`, reconstruct it with Part 1 on a grid running at `oversample * fs`, and measure the amplitude of the f component. Reconstruct only the *middle half* of the record — fine-grid indices from $\frac{N}{4} \cdot \text{oversample}$ up to $\frac{3N}{4} \cdot \text{oversample}$ — because the sum is badly truncated near the two ends. The input amplitude is 1, so the measured amplitude is the gain, and it should be 1.

Starter file — `sincrecon.py`

```
"""The Ideal Filter Does Not Droop: sinc interpolation.

Complete the two functions marked TODO. Do not modify anything below
the divider. Run with: python sincrecon.py
"""

import numpy as np

def sinc_interpolate(samples, fs, t):
    """Whittaker-Shannon reconstruction evaluated at the times in t.

    Sample n sits at time n/fs. Returns an array the same length as t.
    Hint: sinc((t - n*T)/T) is the same as sinc(fs*t - n).
    """
    raise NotImplementedError

def measured_ideal_gain(f, fs, oversample, duration):
    """Amplitude of the f component after ideal reconstruction.
```

```

    Samples  $\cos(2\pi f t)$  at rate  $fs$  with  $N = \text{int}(\text{duration} * fs)$  samples,
    reconstructs on a grid running at  $\text{oversample} * fs$ , but only over the
    middle half (fine indices  $(N/4) * \text{oversample}$  to  $(3N/4) * \text{oversample}$ ),
    then measures the amplitude with tone_amplitude (second parameter
     $\text{oversample} * fs$ ). The input amplitude is 1, so this is the gain.
    """
    raise NotImplementedError

# -----
# Everything below is provided. Do not modify.
# -----

def tone_amplitude(x, fs_fine, f):
    """Amplitude of the component of x at frequency f, via the DFT."""
    n = len(x)
    spectrum = np.fft.rfft(x)
    freqs = np.fft.rfftfreq(n, 1 / fs_fine)
    return 2 * np.abs(spectrum[int(np.argmax(np.abs(freqs - f)))] / n

SAMPLE_RATE = 1000      # Hz
OVERSAMPLE = 8          # fine-grid steps per sample period
DURATION = 1.0          # seconds
TOLERANCE = 1e-5

TEST_FREQS = [50, 100, 200, 300, 450]  # Hz

def main():
    print(f"{'f (Hz)':>8} {'f/fs':>7} {'ideal':>9} {'measured':>10} "
          f"{'|error|':>10} {'ZOH':>8} {'linear':>8}")
    print("-" * 64)

    failures = 0
    for f in TEST_FREQS:
        measured = measured_ideal_gain(f, SAMPLE_RATE, OVERSAMPLE, DURATION)
        error = abs(1.0 - measured)

        failures += error >= TOLERANCE
        zoh = abs(np.sinc(f / SAMPLE_RATE))
        foh = np.sinc(f / SAMPLE_RATE) ** 2
        print(f"{'f':>8} {'f / SAMPLE_RATE':>7.2f} {'1.0:>9.4f} {'measured:>10.4f} "
              f"{'error:>10.2e} {'zoh:>8.4f} {'foh:>8.4f}")

    print("-" * 64)
    if failures:
        print(f"{failures} of {len(TEST_FREQS)} case(s) were not flat.")
    else:
        print("The ideal filter passes every frequency at full amplitude.")

if __name__ == "__main__":
    main()

```

Expected output

f (Hz)	f/fs	ideal	measured	error	ZOH	linear

50	0.05	1.0000	1.0000	2.95e-09	0.9959	0.9918
100	0.10	1.0000	1.0000	3.28e-09	0.9836	0.9675
200	0.20	1.0000	1.0000	5.19e-09	0.9355	0.8751
300	0.30	1.0000	1.0000	1.29e-08	0.8584	0.7368
450	0.45	1.0000	1.0000	6.17e-07	0.6986	0.4881

The ideal filter passes every frequency at full amplitude.

Answer

```
def sinc_interpolate(samples, fs, t):
    n = np.arange(len(samples))
    return np.sinc(fs * np.asarray(t)[: , None] - n) @ np.asarray(samples)

def measured_ideal_gain(f, fs, oversample, duration):
    N = int(duration * fs)
    samples = np.cos(2 * np.pi * f * np.arange(N) / fs)
    lo, hi = N // 4, 3 * N // 4
    t = np.arange(lo * oversample, hi * oversample) / (oversample * fs)
    return tone_amplitude(sinc_interpolate(samples, fs, t),
                          oversample * fs, f)
```

How the one-line interpolation works. `t[: , None]` turns the times into a column, so `fs * t[: , None] - n` is a table with one row per output time and one column per sample, holding $f_s t - n$ in every cell. Applying `np.sinc` gives the weight each sample contributes to each output time, and the matrix product `@ samples` does the multiply-and-add for every output time at once. If you prefer a loop, the same thing is `sum(samples[k] * np.sinc(fs*t - k) for k in range(len(samples)))` — slower but identical.

Why the middle half only. The formula needs samples stretching to infinity in both directions, and the sinc tails die off slowly (like $1/t$). Near the start of a finite record there are no samples to the left, so the sum is missing real contributions and the reconstruction is badly wrong — the error at the very edge is about 0.95, almost the full amplitude of the signal. Half a record away from either end the missing tails have shrunk enough that the error drops to under 10^{-3} . Trimming to the middle half is admitting the filter is unrealisable, not hiding a bug.

Why the reported error is so much smaller than 10^{-3} . The left-over truncation shows up as a small ripple spread across many frequencies, not as a change in amplitude at f itself. `tone_amplitude` reads one DFT bin, so it sees the height of the tone and ignores most of the ripple — which is why the error column sits near 10^{-9} rather than 10^{-3} .

The last two columns are the point. Side by side at $f/f_s = 0.45$: the ideal filter passes 1.0000, the zero-order hold passes 0.6986, and linear interpolation passes 0.4881. That is the comparison on slide 36, in numbers.

5 Assignment F — Straight Lines Droop Twice as Hard

Background

Linear interpolation, also called the first-order hold, joins neighbouring samples with straight lines instead of holding each one flat. The picture is visibly smoother than the staircase, and the reason is visible in the frequency domain too.

Its impulse response $h_1(t)$ is a triangle spanning $[-T, T]$ with peak 1, and a triangle is what you get when you convolve two rectangles of width T . Since convolving in time multiplies in frequency, and one rectangle gives a sinc, the triangle gives a sinc *squared*:

$$H_1(j\omega) = T \operatorname{sinc}^2\left(\frac{\omega T}{2\pi}\right), \quad \text{so} \quad \text{gain}(f) = \operatorname{sinc}^2\left(\frac{f}{f_s}\right).$$

Squaring a number below 1 makes it smaller, so linear interpolation droops *more* in the passband than the zero-order hold. It is still the better filter overall, because outside the passband the same squaring crushes the leaked copies much harder — it is the leakage, not the droop, that decides which one sounds better.

Your task

The starter file contains two functions with their bodies removed. Fill in both. `main()` and `tone_amplitude()` are provided; do not modify them.

Part 1 — `foh_gain(f, fs)` Return the predicted gain $\operatorname{sinc}^2(f/f_s)$.

Part 2 — `measured_foh_gain(f, fs, upsample, duration)` Sample $\cos(2\pi ft)$ at rate `fs`, join the samples with straight lines on a grid running at `upsample * fs`, and measure the amplitude of the f component. Use `np.interp` for the straight lines. Take $N + 1$ samples rather than N , so that the last segment has a sample to lean on at both ends.

Starter file — `linterp.py`

```
"""Straight Lines Droop Twice as Hard: first-order hold gain.

Complete the two functions marked TODO. Do not modify anything below
the divider. Run with:  python linterp.py
"""

import numpy as np

def foh_gain(f, fs):
    """Predicted linear-interpolation gain at frequency f, sampling at fs.

    gain = sinc(f / fs) ** 2
    """
    raise NotImplementedError

def measured_foh_gain(f, fs, upsample, duration):
    """Gain of linear interpolation, measured rather than predicted.

    Samples cos(2*pi*f*t) at rate fs, then joins neighbouring samples
    with straight lines on a grid running at upsample*fs (use np.interp),
    """
```

```

and returns the amplitude of the f component of the result (using
tone_amplitude, second parameter upsample * fs).

Take N+1 coarse samples with N = int(duration*fs), so the final
segment has a sample at both ends; evaluate on N*upsample fine points.

The input amplitude is 1, so this amplitude is the gain.
"""
raise NotImplementedError

# -----
# Everything below is provided. Do not modify.
# -----

def tone_amplitude(x, fs_fine, f):
    """Amplitude of the component of x at frequency f, via the DFT."""
    n = len(x)
    spectrum = np.fft.rfft(x)
    freqs = np.fft.rfftfreq(n, 1 / fs_fine)
    return 2 * np.abs(spectrum[int(np.argmin(np.abs(freqs - f)))] / n

SAMPLE_RATE = 1000      # Hz
UPSAMPLE = 100          # fine-grid steps per sample period
DURATION = 0.1          # seconds
TOLERANCE = 1e-3

TEST_FREQS = [50, 100, 200, 300, 450]  # Hz

def main():
    print(f"{'f (Hz)':>8} {'f/fs':>7} {'predicted':>11} {'measured':>10} "
          f"{'|error|':>10} {'droop':>10}")
    print("-" * 60)

    failures = 0
    for f in TEST_FREQS:
        predicted = foh_gain(f, SAMPLE_RATE)
        measured = measured_foh_gain(f, SAMPLE_RATE, UPSAMPLE, DURATION)
        error = abs(predicted - measured)

        failures += error >= TOLERANCE
        droop_db = 20 * np.log10(predicted)
        print(f"{'f':>8} {'f / SAMPLE_RATE':>7.2f} {'predicted':>11.4f} "
              f"{'measured':>10.4f} {'error':>10.2e} {'droop_db':>7.2f} dB")

    print("-" * 60)
    if failures:
        print(f"{failures} of {len(TEST_FREQS)} case(s) disagree by more than "
              f"{TOLERANCE:g}.")
    else:
        print("Prediction matches measurement at every frequency.")

if __name__ == "__main__":
    main()

```

Expected output

f (Hz)	f/fs	predicted	measured	error	droop
50	0.05	0.9918	0.9918	8.16e-07	-0.07 dB
100	0.10	0.9675	0.9675	3.18e-06	-0.29 dB
200	0.20	0.8751	0.8752	1.15e-05	-1.16 dB
300	0.30	0.7368	0.7369	2.18e-05	-2.65 dB
450	0.45	0.4881	0.4881	3.25e-05	-6.23 dB

Prediction matches measurement at every frequency.

Answer

```
def foh_gain(f, fs):
    return float(np.sinc(f / fs) ** 2)

def measured_foh_gain(f, fs, upsample, duration):
    N = int(duration * fs)
    t_coarse = np.arange(N + 1) / fs
    samples = np.cos(2 * np.pi * f * t_coarse)
    t_fine = np.arange(N * upsample) / (upsample * fs)
    ramp = np.interp(t_fine, t_coarse, samples)
    return tone_amplitude(ramp, upsample * fs, f)
```

Why `np.interp` is linear interpolation. Given the coarse times and their values, `np.interp` returns, for each fine time, the value on the straight line joining the two surrounding coarse samples. That is precisely joining the dots, which is the first-order hold. Compare with `zoh.py`: there `np.repeat` held each value flat, here `np.interp` ramps between them — one word of difference, and the gain goes from sinc to sinc squared.

Why $N + 1$ samples and not N . A straight line needs a point at each end. With only N samples the last segment, from sample $N - 1$ to the end of the record, has nothing on its right, and `np.interp` would hold it flat instead of ramping — a small piece of staircase contaminating the measurement. Taking one extra sample gives that segment its right-hand end. (The test frequencies all complete a whole number of cycles in 0.1 s, so sample N happens to equal sample 0 anyway.)

Comparing with the staircase. At $f/f_s = 0.45$ the zero-order hold gives 0.6986 (−3.11 dB) and linear interpolation gives 0.4881 (−6.23 dB) — exactly twice the droop in decibels, because squaring a gain doubles its value in dB.

6 Assignment G — The Nyquist Threshold

Background

Everything in this lecture builds to one inequality. Sampling copies the spectrum to every multiple of ω_s ; each copy is $2\omega_M$ wide; and the copies stay apart exactly when

$$\omega_s > 2\omega_M, \quad \text{equivalently} \quad f_s > 2f_{\max}.$$

The quantity $2f_{\max}$ is the *Nyquist rate*. Sample faster than it and the copies are separated, so the ideal low-pass filter can lift out the original one and the signal comes back exactly. Sample slower and the copies overlap; the overlapping parts add together, and no filter can undo a sum.

This assignment checks the theorem end to end. You predict the Nyquist rate for a band-limited signal, then reconstruct that signal from samples taken above and below the rate, and confirm that recovery succeeds exactly when the theorem says it should.

Your task

The starter file contains two functions with their bodies removed. Fill in both. `main()` and `sinc_interpolate()` are provided; do not modify them.

Part 1 — `nyquist_rate(frequencies)` The signal is a sum of cosines at the given frequencies. Return the minimum sampling rate that the sampling theorem requires.

Part 2 — `reconstruction_error(frequencies, amplitudes, fs, duration, oversample)` Build the signal $x(t) = \sum_k A_k \cos(2\pi f_k t)$, sample it at rate `fs` using $t = n/f_s$ for $n = 0 \dots N - 1$ with $N = \text{int}(\text{duration} * \text{fs})$, reconstruct it with the provided `sinc_interpolate` on a grid running at `oversample * fs`, and return the largest absolute difference between the reconstruction and the true $x(t)$.

Reconstruct only the *middle half* of the record — fine-grid indices from $\frac{N}{4} \cdot \text{oversample}$ up to $\frac{3N}{4} \cdot \text{oversample}$ — because the interpolation sum is badly truncated near the two ends.

Starter file — `nyquist.py`

```
"""The Nyquist Threshold.

Complete the two functions marked TODO. Do not modify anything below
the divider. Run with:  python nyquist.py
"""

import numpy as np

def nyquist_rate(frequencies):
    """Minimum sampling rate for a sum of cosines at these frequencies."""
    raise NotImplementedError

def reconstruction_error(frequencies, amplitudes, fs, duration, oversample):
    """Largest error after sampling at fs and reconstructing.

    Builds x(t) = sum_k A_k*cos(2*pi*f_k*t), samples it at rate fs with
    N = int(duration*fs) samples at t = n/fs, reconstructs with
    sinc_interpolate on a grid running at oversample*fs, and compares
```

```

    against the true  $x(t)$ .

    Use only the middle half (fine indices  $(N/4)*oversample$  up to
     $(3*N/4)*oversample$ ); the sum is badly truncated near the ends.
    """
    raise NotImplementedError

# -----
# Everything below is provided. Do not modify.
# -----

def sinc_interpolate(samples, fs, t):
    """Whittaker-Shannon reconstruction evaluated at the times in t."""
    n = np.arange(len(samples))
    return np.sinc(fs * np.asarray(t)[:, None] - n) @ np.asarray(samples)

DURATION = 1.0          # seconds
OVERSAMPLE = 8          # fine-grid steps per sample period
RECOVERED = 1e-2        # error below this counts as recovered

TEST_CASES = [
    # (frequencies Hz, amplitudes, sampling rate Hz)
    ([100, 250], [1.0, 0.5], 800),
    ([100, 250], [1.0, 0.5], 400),
    ([60, 180, 300], [1.0, 0.6, 0.3], 960),
    ([60, 180, 300], [1.0, 0.6, 0.3], 480),
    ([200], [1.0], 640),
    ([200], [1.0], 320),
]

def main():
    print(f"{'tones (Hz)':>14} {'Nyquist':>8} {'fs':>6} {'fs/rate':>8} "
          f"{'max error':>11} {'theorem':>9} {'measured':>10}")
    print("-" * 74)

    failures = 0
    for frequencies, amplitudes, fs in TEST_CASES:
        rate = nyquist_rate(frequencies)
        error = reconstruction_error(frequencies, amplitudes, fs,
                                    DURATION, OVERSAMPLE)

        theorem = fs > rate
        measured = error < RECOVERED
        failures += theorem != measured
        print(f"{'','.join(str(f) for f in frequencies):>14} {rate:>8} "
              f"{'fs:>6} {'fs / rate:>8.2f} {'error:>11.2e} "
              f"{'recover' if theorem else 'alias':>9} "
              f"{'recovered' if measured else 'ALIASED':>10}")

    print("-" * 74)
    if failures:
        print(f"{failures} of {len(TEST_CASES)} case(s) contradict the theorem.")
    else:
        print("Recovery happened exactly when fs exceeded the Nyquist rate.")

if __name__ == "__main__":
    main()

```

Expected output

tones (Hz)	Nyquist	fs	fs/rate	max error	theorem	measured
100,250	500	800	1.60	1.59e-03	recover	recovered
100,250	500	400	0.80	9.27e-01	alias	ALIASED
60,180,300	600	960	1.60	1.68e-03	recover	recovered
60,180,300	600	480	0.80	5.58e-01	alias	ALIASED
200	400	640	1.60	1.33e-03	recover	recovered
200	400	320	0.80	1.85e+00	alias	ALIASED

Recovery happened exactly when fs exceeded the Nyquist rate.

Answer

```
def nyquist_rate(frequencies):
    return 2 * max(frequencies)

def reconstruction_error(frequencies, amplitudes, fs, duration, oversample):
    N = int(duration * fs)
    t_coarse = np.arange(N) / fs
    samples = sum(a * np.cos(2 * np.pi * f * t_coarse)
                  for f, a in zip(frequencies, amplitudes))

    lo, hi = N // 4, 3 * N // 4
    t = np.arange(lo * oversample, hi * oversample) / (oversample * fs)
    truth = sum(a * np.cos(2 * np.pi * f * t)
               for f, a in zip(frequencies, amplitudes))

    return float(np.max(np.abs(sinc_interpolate(samples, fs, t) - truth)))
```

Why Part 1 is just $2 * \max(\text{frequencies})$. The band limit $f_{\max}$ of a sum of cosines is simply its highest frequency — there is nothing above it. The theorem asks for $f_s > 2f_{\max}$, so the rate to report is twice the largest tone. Note it is a strict inequality: sampling exactly *at* the Nyquist rate is not enough, because the copies then touch edge to edge.

Why the same `sum(...)` appears twice. Once at the coarse times to make the samples, once at the fine times to make the thing you compare against. They are the same formula evaluated on two different grids: one is what the converter captured, the other is the truth it was trying to capture.

Reading the two groups of rows. Above the rate the error sits near 1.5×10^{-3} , which is the sinc-truncation floor from assignment E — the reconstruction is as exact as a finite record allows. Below the rate the error jumps to between 0.56 and 1.85, comparable to the size of the signal itself. That is not a slightly worse answer, it is a different signal: the tones above $f_s/2$ folded down, landed on top of the ones below, and the sinc filter faithfully reconstructed the mixture it was handed.

Why the threshold is safe to set at 10^{-2} . The two groups are three orders of magnitude apart, so any cut between about 10^{-2} and 10^{-1} gives the same verdicts. There is no delicate tuning hiding in that constant.

Worth trying once. Change a rate to exactly the Nyquist rate — 500 for the first signal — and watch what happens. The theorem's strict inequality is not a technicality.